

Clasificando la tríada CIA Compra Segura

Pablo Nieto Pérez

Seguridad De Aplicaciones

Tecnología en Desarrollo de Software para Negocios Digitales, Universidad de Manizales

03 de Septiembre de 2026

1. **Introducción:** El propósito de este trabajo es aplicar los principios de la tríada CIA sobre la aplicación nuestra aplicación Compra Segura tomando como base el trabajo #1 ya entregado

2. **Registro de hallazgos:** Los hallazgos principales de los fallos en el sistema vienen principalmente desde la base de datos, esto después de hacer QA lo cual es aseguramiento de calidad del sistema, que es de mi punto de vista el tema más crítico del sistema viene desde la creación de la base de datos, ya que no cuenta con estándares de seguridad básicos, tampoco maneja cultura devops, como por ejemplo una base de datos que declare sus relaciones y sus restricciones y difiere problemas que desde el principio se pueden resolver desde base de datos para que se tengan que hacer desde un back end adicional, algunos de los puntos más críticos del sistema, considero que son:

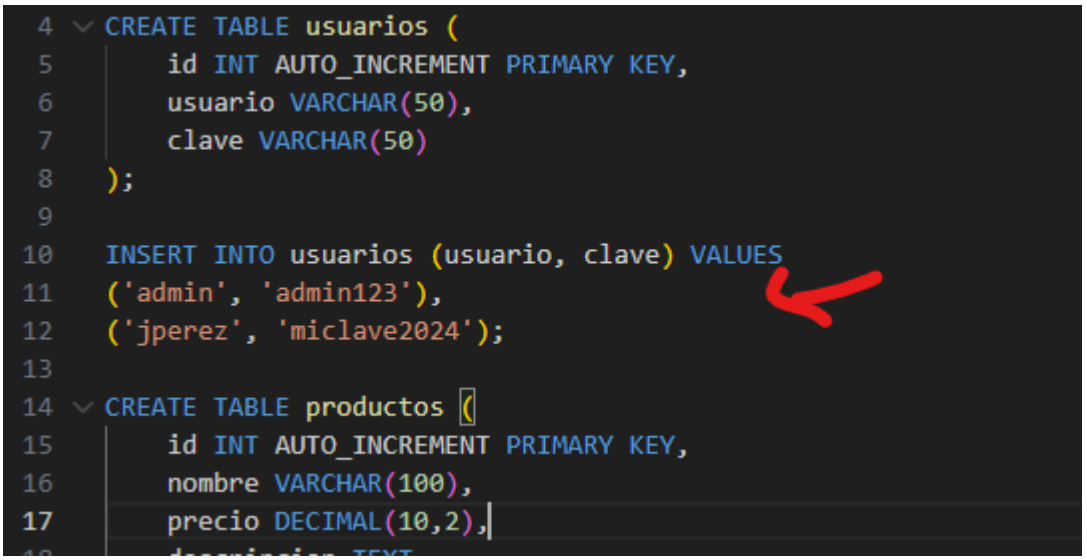
    a. Archivo revisado **compraseguras.sql**

    ¿Qué observamos?

    Empiezo por acá porque para mí el problema empieza desde la base de datos, lo primero que salta es el nombramiento, que se llama compraseguras, y no compra_segura ni compraSegura, pero podemos omitir esta parte ya que el nombramiento de cada base de datos dependía de cada uno, como punto más crítico es que las contraseñas se guardan como texto plano

    ('admin', 'admin123'), ('jperez', 'miclave2024');

    Cualquiera que abra el archivo o entre a phpMyAdmin ve las claves de todos los usuarios



¿Por qué creemos que es un riesgo?

Porque cualquiera que acceda a la base de datos se lleva las contraseñas de todos los usuarios y como la columna es clave VARCHAR(50), entonces tampoco se puede arreglar sin modificar la tabla, como un hash que son 60 caracteres, así que MySQL lo cortaría y nadie podría entrar

Nivel de preocupación: Alto

¿Qué observamos?

Que la tabla usuarios no tiene ninguna columna de roles, solo id, usuario y clave y no existe un es_admin ni nada parecido entonces "admin" es únicamente el nombre de un usuario, no un rol de verdad

¿Por qué creemos que es un riesgo?

Porque si no hay roles el sistema no tiene forma de distinguir entre un cliente y un administrador y cualquier cuenta registrada tiene exactamente los mismos permisos y como tampoco hay llaves foráneas la base acepta pedidos con usuario_id = 0 o con productos que no existen entonces quedan registros huérfanos que no se pueden rastrear

Nivel de preocupación: Alto

- Qué es un hash? Es el resultado de pasar un texto por una función matemática que lo convierte en una cadena de longitud fija

b. Archivo revisado **login.php**

¿Qué observamos? Que lo que el usuario escribe se mete directo dentro de la consulta SQL, sin revisarse antes:



```php
<?php
include 'conexion.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $usuario = $_POST['usuario'];
    $clave   = $_POST['clave'];

    $sql = "SELECT * FROM usuarios WHERE usuario = '$usuario' AND clave = '$clave'";
    $resultado = $conn->query($sql);

    if ($resultado && $resultado->num_rows > 0) {
        $fila = $resultado->fetch_assoc();
        $_SESSION['usuario_id'] = $fila['id'];
        $_SESSION['usuario'] = $fila['usuario'];
        echo "Bienvenido, " . $fila['usuario'] . ". Acceso concedido.";
    } else {
        echo "Usuario o clave incorrectos.";
    }
}
?>
<form method="POST">
  Usuario: <input type="text" name="usuario"><br>
  Clave: <input type="password" name="clave"><br>
  <button type="submit">Ingresar</button>
</form>
```

Ahí $usuario y $clave van pegados dentro del texto de la consulta o sea que lo que escriba el usuario se vuelve parte de la instrucción, porque no hay consultas preparadas en ningún lado

Si por ejemplo en el login escribimos ' OR '1'='1 en el campo usuario y cualquier cosa en la clave, el sistema responde con true entonces da acceso, sin saber ninguna contraseña

¿Por qué creemos que es un riesgo?

Creemos que es un riesgo ya que cualquiera con conocimientos básicos de base de datos, o una IA a la mano junto con malas intenciones, puede ingresar al sistema sin tener una cuenta y no necesita adivinar la contraseña o robársela, solo escribe un texto en el formulario y ya está adentro con la sesión de admin y para el sistema eso fue un login normal

Nivel de preocupación: Alto

c. Archivo revisado pedido.php

¿Qué observamos?

Que el precio del producto viaja desde el navegador, en un campo oculto del formulario

```
13  }
14  ?>
15  <form method="POST">
16  <input type="hidden" name="producto_id" value="1">
17    <input type="hidden" name="precio" value="3000">
18    Cantidad: <input type="number" name="cantidad" value="1">
19    <button type="submit">Confirmar pedido</button>
20  </form>
```

Y el php lo recibe y lo usa tal cual sin verificar nada

```
1   <?php
2   include 'conexion.php';
3   if ($_SERVER['REQUEST_METHOD'] === 'POST') {
4       $producto_id = $_POST['producto_id'];
5       $precio      = $_POST['precio'];
6       $cantidad    = $_POST['cantidad'];
7       $usuario_id  = $_SESSION['usuario_id'] ?? 0;
8       $total = $precio * $cantidad;
9
10      $sql = "INSERT INTO pedidos (usuario_id, producto_id, cantidad
11      $conn->query($sql);
12      echo "Pedido registrado. Total a pagar: $" . $total;
13  }
14  ?>
```

O sea que el sistema le está preguntando al cliente cuánto vale el producto y con las herramientas de desarrollador del navegador uno cambia ese 3000 por un 1 y envía el formulario, entonces el pedido queda guardado con ese total

Lo mismo con cualquier valor, como si el cliente quiere poner 0 o -1000

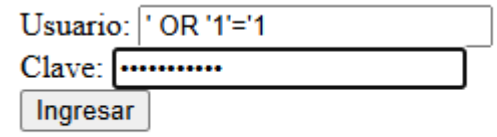
¿Por qué creemos que es un riesgo?

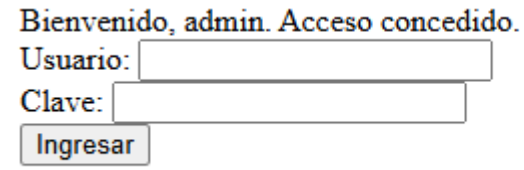
Porque el cliente decide lo que paga, el precio real está en la tabla productos, pero el código nunca lo consulta entonces no hay nada con qué comparar y el total falso queda guardado en la base como si fuera de verdad, como tampoco hay control de sesiones y vimos que cualquiera puede ingresar, esos pedidos se pueden hacer sin haber iniciado sesión y quedan a nombre del usuario 0 que no existe

Nivel de preocupación: Alto

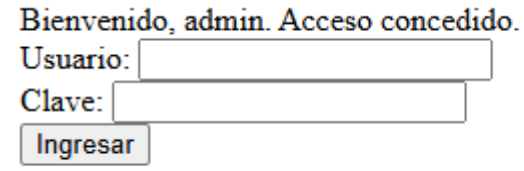
3. Reto

    a. Funcionó el bypass? Sí, escribimos ' OR '1'='1 en usuario y cualquier texto en clave y el sistema respondió "Bienvenido, admin. Acceso concedido” y nunca usamos una contraseña real





    b. ¿Qué pudiste ver una vez dentro? El sistema nos identificó como admin y guardó su id y su nombre en la sesión, desde ahí cualquier función como registrar un pedido, queda asociada a esa cuenta aunque nunca nos autenticamos



    c. ¿Por qué ocurre en el código y no en la base de datos? Porque la base de datos hace lo que le pidieron que es recibe una consulta ya armada y la ejecuta y el error está en login.php que arma esa consulta pegando texto del usuario dentro del SQL en vez de mandar el dato aparte

4. Tabla de clasificación CIA

    a. Hallazgo de compraseguras.sql debido a las claves en texto plano

    Qué observamos? Que las contraseñas se guardan sin cifrar

    C: Sí
    I: No
    A: No

Justificación: Porque cualquiera con acceso a la base de datos puede leer admin123 y miclave2024 directo del INSERT: se expone un secreto

C: se expone un secreto (cualquiera lee admin123 y miclave2024 del INSERT)
I: no se altera ningún dato
A: no se cae el servicio

b. Hallazgo en login.php por inyección sql, que se puede hacer bypass

Qué observamos? Que lo que el usuario escribe se mete directo en la consulta sql, sin consultas preparadas

C: Sí
I: Sí
A: Sí

Justificación:

C: con ' OR '1'='1 se entra sin conocer ninguna clave, entonces se puede hacer un acceso no autorizado a la información y a la sesión de admin

I: la misma inyección permite meter comandos como actualizar información o borrarla y modificar datos que no ser deberían tocar

A: la misma inyección permite borrar tablas y dejar el sistema sin datos y sin funcionar

c. Hallazgo de pedido.php por manipulación del precio

Qué observamos? Que el precio viaja en un campo oculto y el php lo recibe y lo usa tal cual sin verificar nada

C: No
I: Sí
A: No

Justificación:

C: no se expone ningún secreto
I: cambié el value="3000" a “-10” desde F12 y se guardó un total falso en la tabla pedidos entonces el precio real está en la tabla productos, pero el código nunca lo consulta para comparar
A: no se cae el servicio

d. Hallazgo de cookie “PHPSESSID” sin protección

   Qué observamos? Que el identificador de sesión PHPSESSID viaja sin las banderas HttpOnly ni Secure

   C: Sí
   I: No
   A: No

   Justificación:

   C: si un atacante lee o roba el la cookie que es visible revisando panel del en Cookies, puede secuestrar la sesión y hacerse pasar por el usuario sin conocer su clave
   I: por sí mismo no altera ningún dato
   A: no se cae el servicio

5. Respuestas a las 2 preguntas de discusión

   a. Algún hallazgo rompe más de un principio?

      Sí, por ejemplo login.php es uno que rompe más de un principio y es el de con la inyección SQL, y ese rompe los tres a la vez, la confidencialidad porque con ' OR '1'='1 uno entra sin saber ninguna clave y queda con acceso a la información y a la sesión de admin, también la integridad porque esa misma inyección deja meter comandos para actualizar o borrar datos que uno no debería tocar y también la disponibilidad porque con esa misma falla se pueden borrar tablas completas y dejar el sistema sin datos y sin funcionar, yo por eso considero que es el más peligroso porque con una sola falla se abre la puerta para todo

   b. Si solo pudieras corregir uno esta semana, cuál?

      Yo arreglaría primero el tema de login.php porque es el único que rompe los tres principios a la vez y el más fácil de explotar, entonces cualquiera escribe ' OR '1'='1 y ya está adentro sin saber ninguna contraseña, con consultas preparadas se cierra de una la puerta más grande del sistema

      Además es lo más grave para el negocio porque por ahí se roba la sesión de admin, se alteran o borran datos y se registran pedidos falsos, entonces ese es el que más protege el negocio

6. Desafío

Paso 1 (Confidencialidad): Todo empieza con las contraseñas en texto plano de compraseguras.sql. Un atacante llega a la base de datos, ya sea con la inyección SQL de login.php o abriendo el archivo o phpMyAdmin, y como las claves están guardadas sin cifrar (clave VARCHAR(50)) lee de una admin / admin123 sin tener que descifrar nada, o sea que se lleva la contraseña real del administrador.

Paso 2 (Autenticidad): Con esa clave real ya ni necesita hacer trampa, entra al sistema como el admin de verdad y el sistema ni cuenta se da, porque no hay roles ni doble verificación, solo guarda el id y el nombre en la sesión, entonces para el sistema es un login normal y le cree que es el administrador.

Paso 3 (Integridad): Ya adentro como admin el atacante empieza a dañar los datos, cambia precios, modifica o borra pedidos y hasta le cambia la clave al otro usuario, entonces la información deja de ser confiable porque cualquiera pudo haberla alterado.

Paso 4 (Disponibilidad): Y el golpe final, como tiene control total corre un DROP TABLE y borra las tablas de usuarios, pedidos y productos, o le cambia la clave al admin y deja a todos por fuera, entonces ya nadie puede entrar ni usar Compra Segura y como no había copias de seguridad el sistema se cae completo. Así un problema que empezó siendo solo de confidencialidad terminó rompiendo la disponibilidad de todo el sistema.

Bibliografías

https://www.php.net/manual/es/mysqli.quickstart.prepared-statements.php

https://www.php.net/manual/es/function.password-hash.php

https://stackoverflow.com/questions/1370951/what-is-phpsessid